

18. Battery MHE

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	3
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	4
5.3 import 群の詳細	4
6 このファイルを読む前に必要な基礎知識	5
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	5
6.2 名前、代入、型、参照	5
6.3 関数、引数、戻り値、スコープ	6
6.4 module、package、import、console_scripts	6
6.5 例外、try/finally、with、資源解放	6
6.6 class、独自クラス、インスタンス、self、継承	6
6.7 先頭アンダースコア、__init__、名前付けの作法	7
6.8 デコレータと @dataclass	7
6.9 list、dict、deque、NumPy 配列、pandas DataFrame	8
6.10 目的関数、制約、L-BFGS-B、SHGO、有限 grid 証明	8
6.11 車両力学、電力収支、効率 map	8
6.12 SoC、内部抵抗、端子電圧、温度、MHE	9
6.13 制御、状態、入力、モデル予測制御 MPC	9
6.14 freshness、filter、guard、fallback、fail-safe	9

7	関数・クラスを上から順に解説	9
7.1	L11 クラス MheInput	9
7.2	L24 クラス MheMeas	11
7.3	L31 クラス BatteryMHE	11
7.4	L32 関数 BatteryMHE.__init__	13
7.5	L54 関数 BatteryMHE.push	14
7.6	L57 関数 BatteryMHE._simulate	14
7.7	L85 関数 BatteryMHE.estimate	16
7.8	L89 関数 BatteryMHE.estimate.cost	17
8	処理の流れ	18
9	このファイルの位置づけ	18

1 対象

項目	内容
ファイル	mpc_solarcar/estimator.py
ソース SHA-256	a5b46d1669a9ef838d07a4747d013e8c11bf0c4080cef73f07d0cd4fcadc8d31
種別	Python
区分	estimator
役割	観測された I/V/SoC/Tb と model を使って内部 SoC/Tb を短ホライズンで補正する。
起動文脈	mpc_node 内の状態推定器。

2 このファイルを読む理由

観測された I/V/SoC/Tb と model を使って内部 SoC/Tb を短ホライズンで補正する。

- BatteryMHE が入力列を保持し、最尤に近い初期状態を逆推定する。
- planner 本体の物理モデルをそのまま使う。

3 呼び出し関係

3.1 どこから呼ばれるか

- mpc_node.py

3.2 次に読むと理解が進むファイル

- mpc_solarcar/model.py

3.3 同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
@dataclass
class MheInput:
    v_ms: float
    slope_pct: float
    G_poa: float
    Tcell_C: float
    Tamb_C: float
    headwind_ms: float
    dt: float
    inertial_power_w: float = 0.0
```

```
elevation_m: float = 0.0
```

```
@dataclass
class MheMeas:
    soc: Optional[float] = None
    Tb: Optional[float] = None
    I: Optional[float] = None
    V: Optional[float] = None
```

5 主要構造の読み取り

主要クラスは MheInput, MheMeas, BatteryMHE。主要関数は push, estimate, cost。

5.1 主要クラス・関数

行	種別	名前	補足
11	class	MheInput	
24	class	MheMeas	
31	class	BatteryMHE	
32	function	__init__	args: self, model, horizon_steps, w_soc, w_tb, w_i, w_v, w_prior, soc_bounds, tb_bounds
54	function	push	args: self, u, y
57	function	_simulate	args: self, z0, Tb0
85	function	estimate	args: self, z_init, Tb_init
89	function	cost	args: x

5.2 ファイルを上から読んだときの定義順

行	内容
11	クラス MheInput を定義する。
24	クラス MheMeas を定義する。
31	クラス BatteryMHE を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	import math	Python 標準のスカラー数値関数と定数を使うため。実際に参照する名前と行は使用位置欄で確認する。このファイル内での主な使用位置は L94, L96, L98, L100。
2	from collections import deque	固定長の時系列や遅延キューを効率よく保持するため。このファイル内での主な使用位置は L45。
3	from dataclasses import dataclass	dataclasses から dataclass を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L10, L23。

4	<code>from typing import Optional, Tuple</code>	typing から Optional, Tuple を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L25, L26, L27, L28, L41, L42, L85。
6	<code>import numpy as np</code>	数値配列、clip、補間、統計計算を行うため。このファイル内での主な使用位置は L104。
7	<code>from scipy.optimize import minimize</code>	目的関数と制約・bounds に基づく連続数値最適化を解くため。このファイル内での主な使用位置は L106。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Python インタプリタが読み込む -> OS 上のプロセス -> Python オブジェクトをメモリに生成 -> 関数や callback を実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# node と alias は同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):  
    return min(maximum, max(minimum, value))  
  
v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが‘self.z’を更新すればオブジェクトの状態が残るが、単に‘z’を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の‘console_scripts’は、端末で使う実行可能名と‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->  
main()を呼び
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 self として ‘__init__’ へ渡す。‘self’ は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

counter = Counter()
counter.increment()
```

‘super().__init__(...)’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体が作られず、‘create_publisher’などを正しく使えない。

根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.7 先頭アンダースコア、__init__、名前付けの作法

‘self.step_solar’の先頭 1 個のアンダースコアは「クラス内部で使う実装詳細」という慣習であり、アクセスを強制禁止する機構ではない。外部コードから呼べるが、公開 API として依存しない意思表示である。

‘__init__’の前後 2 個のアンダースコアは Python が定めた特殊メソッド名である。クラス生成時に自動で呼ばれる。任意の名前に前後 2 個を付けて独自仕様を作ることは避ける。

‘_’で始まるローカル変数は、未使用または外部へ見せない意図を示す場合がある。例えば ‘_base_csv’は戻り値として受け取る必要はあるが、その後の処理では使わないことを読者へ示す。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.8 デコレータと @dataclass

‘@名前’は、直後に定義した関数またはクラスを別の関数へ渡し、その結果で定義名を置き換えるデコレータ構文である。‘@Class’という一般構文があるのではなく、‘@dataclass’など実際のデコレータ名を書く。

‘@dataclass’は型注釈付きフィールドから ‘__init__’、‘__repr__’、比較処理などを自動生成する。物理パラメータや解析結果のように、名前付きデータを一まとまりで運ぶ用途に向く。

```
from dataclasses import dataclass

@dataclass
class State:
    soc: float
    temperature_c: float

state = State(soc=0.8, temperature_c=30.0)
```

自動生成は検証を自動で保証しない。単位、許容範囲、相互依存制約は ‘__post_init__’ や別の検証関数で確認する必要がある。

根拠資料

- [Python 公式ライブラリ: dataclasses](#)
- [Python 公式チュートリアル: Classes](#)

6.9 list、dict、deque、NumPy 配列、pandas DataFrame

list は順序付き可変列、tuple は順序付きで通常変更しない列、dict はキーから値を引く対応表、deque は両端追加・削除が効率的なキューである。どの構造を選ぶかはアクセス方法と更新方法で決まる。

NumPy 配列は同種数値を連続的に扱い、要素ごとの演算、clip、補間、線形代数を簡潔に書く。shape は各次元の要素数であり、速度系列なら通常 ‘(N,)’、候補集団なら ‘(population, N)’ となる。

pandas DataFrame は列名を持つ表である。CSV 読込後は列型、欠損、単位、timezone、並び順、重複を明示的に処理しなければ、数値計算が動いても意味が誤る。

6.10 目的関数、制約、L-BFGS-B、SHGO、有限 grid 証明

数値最適化器は、利用者が与えた目的関数を複数の候補点で評価し、より小さい値を持つ候補を探す。solver が物理を理解するのではなく、物理と運用価値は cost 関数へ書かれる。

L-BFGS-B は変数ごとの上下限を扱える局所最適化法である。初期値の近くの谷へ収束し得るため、非凸問題では複数 seed や大域探索と組み合わせる。success が False でも有限な候補が返る場合があるため、採用条件をコード側で決める。

SHGO は定めた sampling と局所最適化を組み合わせる大域最適化法である。有限 Cartesian grid の全列挙は、その grid 上の最良を証明できるが、連続領域全体の最良を自動的に証明しない。資料ではこの証明範囲を区別する。

根拠資料

- [SciPy 公式: scipy.optimize.minimize](#)
- [SciPy 公式: scipy.optimize.shgo](#)

6.11 車両力学、電力収支、効率 map

$$F_{\text{aero}} = \frac{1}{2} \rho C_d A (v + v_{\text{wind}})^2, \quad F_{\text{roll}} \approx mg C_{rr} \cos \theta, \quad F_{\text{grade}} = mg \sin \theta$$

車輪機械 power は概ね ‘P_mech = F_total * v + P_inertia’ で、駆動時と回生時で異なる効率 map を通して DC 側 power へ変換する。map は速度・torqueなどを軸にした実測または同定 table であり、範囲外の補間・clip 規則もモデルの一部である。

$$P_{\text{pack}} = P_{\text{drive,dc}} - P_{\text{regen,dc}} + P_{\text{aux}} - P_{\text{pv}}$$

符号規約は必ずコードで確認する。本プロジェクトでは正の pack power を放電側として SoC を減らす処理が中心であり、発電と回生は pack 負荷を下げる方向に働く。

6.12 SoC、内部抵抗、端子電圧、温度、MHE

$$V = V_{oc}(z, T) - IR_{total}(z, T, I), \quad z_{k+1} = z_k - \frac{\eta(I, T)I\Delta t}{Q_{eff}}$$

SoC は直接完全には観測できないため、電流積算、OCV、端子電圧、温度、容量、効率を組み合わせる。内部抵抗は SoC、温度、電流方向で変わり、発熱と電圧制約の両方へ影響する。

MHE は有限時間窓の状態と観測誤差をまとめて最適化し、SoC や温度を推定する。古い測定や欠損値を同じ重みで使うと推定が壊れるため、timestamp freshness と観測可用性を入口で確認する。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)

6.13 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ' $x[k+1] = f(x[k], u[k], w[k])$ ' と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが receding horizon である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)

6.14 freshness、filter、guard、fallback、fail-safe

分散システムでは最後に受け取った値が現在も有効とは限らない。受信時刻と timeout から freshness を判定し、stale 値を計画状態へ無条件に同期しない。

filter は noise と一時的な飛び値を抑えるが、遅れを生む。slew limit は指令変化率を制限する。安全 guard は solver の cost 罰則とは別に、現在出力へ強制制約を適用する最後の防波堤である。

fallback は失敗時の代替動作を事前に決める設計である。前回計画保持、物理に基づく決定論的入力、停止、低速制限などから、故障 mode ごとに選ぶ。fallback 発生は status と log へ残し、正常解と区別する。

7 関数・クラスを上から順に解説

7.1 L11 クラス MheInput

- 行範囲: L11–L20
- 所属: module 直下
- 定義: `MheInput(bases=None)`

- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- class 文は新しい型を定義する。丸括弧内は継承する基底クラスである。
- @ で始まる行は、定義したクラスを加工する decorator である。

上から順の処理

1. v_ms にを代入する。
2. slope_pct にを代入する。
3. G_poa にを代入する。
4. Tcell_C にを代入する。
5. Tamb_C にを代入する。
6. headwind_ms にを代入する。
7. dt にを代入する。
8. inertial_power_w に 0.0 を代入する。
9. elevation_m に 0.0 を代入する。

代表コード断片

```
class MheInput:
    v_ms: float
    slope_pct: float
    G_poa: float
    Tcell_C: float
    Tamb_C: float
    headwind_ms: float
    dt: float
    inertial_power_w: float = 0.0
    elevation_m: float = 0.0
```

7.2 L24 クラス MheMeas

- 行範囲: L24–L28
- 所属: module 直下
- 定義: MheMeas(bases=None)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無いか、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- class 文は新しい型を定義する。丸括弧内は継承する基底クラスである。
- @ で始まる行は、定義したクラスを加工する decorator である。

上から順の処理

1. soc に None を代入する。
2. Tb に None を代入する。
3. I に None を代入する。
4. V に None を代入する。

代表コード断片

```
class MheMeas:
    soc: Optional[float] = None
    Tb: Optional[float] = None
    I: Optional[float] = None
    V: Optional[float] = None
```

7.3 L31 クラス BatteryMHE

- 行範囲: L31–L111
- 所属: module 直下
- 定義: BatteryMHE(bases=None)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無いか、return 文が明示されていない。

- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- class 文は新しい型を定義する。丸括弧内は継承する基底クラスである。

上から順の処理

1. 関数 `__init__` を定義する。
2. 関数 `push` を定義する。
3. 関数 `_simulate` を定義する。
4. 関数 `estimate` を定義する。

代表コード断片

```
class BatteryMHE:
    def __init__(
        self,
        model,
        horizon_steps: int = 12,
        w_soc: float = 50.0,
        w_tb: float = 5.0,
        w_i: float = 1.0,
        w_v: float = 1.0,
        w_prior: float = 5.0,
        soc_bounds: Tuple[float, float] = (0.05, 0.98),
        tb_bounds: Tuple[float, float] = (-10.0, 65.0),
    ):
        self.model = model
        self.samples = deque(maxlen=max(2, int(horizon_steps)))
        self.w_soc = float(w_soc)
        self.w_tb = float(w_tb)
        self.w_i = float(w_i)
        self.w_v = float(w_v)
        self.w_prior = float(w_prior)
        self.soc_bounds = soc_bounds
        self.tb_bounds = tb_bounds

    def push(self, u: MheInput, y: MheMeas):
        self.samples.append((u, y))

    def _simulate(self, z0: float, Tb0: float):
        z = float(z0)
        Tb = float(Tb0)
        outputs = []
        for (u, _) in self.samples:
            out = self.model.electrical_balance(
                u.v_ms,
                u.slope_pct,
                z,
            )
        ...
```

7.4 L32 関数 BatteryMHE.__init__

- 行範囲: L32–L52
- 所属: BatteryMHE
- 定義: `__init__(self, model, horizon_steps: int = 12, w_soc: float = 50.0, w_tb: float = 5.0, w_i: float = 1.0, w_v: float = 1.0, w_prior: float = 5.0, soc_bounds: Tuple[float, float] = (0.05, 0.98), tb_bounds: Tuple[float, float] = (-10.0, 65.0))`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `deque`, `float`, `int`, `max`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: `self.model`, `self.samples`, `self.soc_bounds`, `self.tb_bounds`, `self.w_i`, `self.w_prior`, `self.w_soc`, `self.w_tb`, `self.w_v`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self.model` に `model` の結果を代入する。
2. `self.samples` に `deque(maxlen=max(2, int(horizon_steps)))` の結果を代入する。
3. `self.w_soc` に `float(w_soc)` の結果を代入する。
4. `self.w_tb` に `float(w_tb)` の結果を代入する。
5. `self.w_i` に `float(w_i)` の結果を代入する。
6. `self.w_v` に `float(w_v)` の結果を代入する。
7. `self.w_prior` に `float(w_prior)` の結果を代入する。
8. `self.soc_bounds` に `soc_bounds` の結果を代入する。
9. `self.tb_bounds` に `tb_bounds` の結果を代入する。

代表コード断片

```
def __init__(
    self,
    model,
    horizon_steps: int = 12,
    w_soc: float = 50.0,
    w_tb: float = 5.0,
    w_i: float = 1.0,
    w_v: float = 1.0,
    w_prior: float = 5.0,
    soc_bounds: Tuple[float, float] = (0.05, 0.98),
```

```

        tb_bounds: Tuple[float, float] = (-10.0, 65.0),
    ):
        self.model = model
        self.samples = deque(maxlen=max(2, int(horizon_steps)))
        self.w_soc = float(w_soc)
        self.w_tb = float(w_tb)
        self.w_i = float(w_i)
        self.w_v = float(w_v)
        self.w_prior = float(w_prior)
        self.soc_bounds = soc_bounds
        self.tb_bounds = tb_bounds

```

7.5 L54 関数 BatteryMHE.push

- 行範囲: L54–L55
- 所属: BatteryMHE
- 定義: `push(self, u: MheInput, y: MheMeas)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `append`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self.samples`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self.samples.append(...)` を実行する。

代表コード断片

```

def push(self, u: MheInput, y: MheMeas):
    self.samples.append((u, y))

```

7.6 L57 関数 BatteryMHE._simulate

- 行範囲: L57–L83
- 所属: BatteryMHE
- 定義: `_simulate(self, z0: float, Tb0: float)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `append`, `electrical_balance`, `float`, `soc_step`

- 戻り値の要点: (outputs, z, Tb)
- 主なローカル代入名: I, P_pack, Tb, Tb_next, V, _, dt, loss_int, out, outputs, u, z, z_next
- 読み取る主なインスタンス属性: self.model, self.samples
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 1、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. z に float(z0) の結果を代入する。
2. Tb に float(Tb0) の結果を代入する。
3. outputs に [] の結果を代入する。
4. self.samples を順に走査し、各要素を (u, _) に入れて処理する。
5. out に self.model.electrical_balance(u.v_ms, u.slope_pct, z, Tb, u.G_poa, u.Tcell_C, headwind_ms=u.headwind_ms, inertial_power_w=u.inertial_power_w, ambient_temp_c=u.Tamb_C, elevation_m=u.elevation_m) の結果を代入する。
6. I に float(out['I']) の結果を代入する。
7. V に float(out['V']) の結果を代入する。
8. P_pack に float(out['P_pack']) の結果を代入する。
9. loss_int に float(out['losses_int']) の結果を代入する。
10. dt に float(u.dt) の結果を代入する。
11. z_next に self.model.soc_step(z, P_pack, dt) の結果を代入する。
12. Tb_next に $Tb + dt / 1800.0 * (u.Tamb_C - Tb) + loss_int * dt / 50000.0$ の結果を代入する。
13. outputs.append(...) を実行する。
14. (z, Tb) に (z_next, Tb_next) の結果を代入する。
15. (outputs, z, Tb) を返す。

代表コード断片

```
def _simulate(self, z0: float, Tb0: float):
    z = float(z0)
    Tb = float(Tb0)
    outputs = []
    for (u, _) in self.samples:
        out = self.model.electrical_balance(
            u.v_ms,
            u.slope_pct,
            z,
            Tb,
            u.G_poa,
            u.Tcell_C,
```

```

        headwind_ms=u.headwind_ms,
        inertial_power_w=u.inertial_power_w,
        ambient_temp_c=u.Tamb_C,
        elevation_m=u.elevation_m,
    )
    I = float(out['I'])
    V = float(out['V'])
    P_pack = float(out['P_pack'])
    loss_int = float(out['losses_int'])
    dt = float(u.dt)
    z_next = self.model.soc_step(z, P_pack, dt)
    Tb_next = Tb + (dt / 1800.0) * (u.Tamb_C - Tb) + (loss_int * dt) / 50000.0
    outputs.append((z_next, Tb_next, I, V))
    z, Tb = z_next, Tb_next
return outputs, z, Tb

```

7.7 L85 関数 BatteryMHE.estimate

- 行範囲: L85–L111
- 所属: BatteryMHE
- 定義: `estimate(self, z_init:float, Tb_init:float)->Tuple[float, float]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_simulate`, `array`, `dict`, `float`, `isfinite`, `len`, `minimize`, `zip`
- 戻り値の要点: `(float(zN), float(TbN)) / (z_init, Tb_init) / J / (z_init, Tb_init)`
- 主なローカル代入名: `I_pred`, `J`, `Tb0`, `TbN`, `Tb_pred`, `V_pred`, `_`, `bounds`, `meas`, `outputs`, `res`, `x0`, `z0`, `zN`, `z_pred`
- 読み取る主なインスタンス属性: `self._simulate`, `self.samples`, `self.soc_bounds`, `self.tb_bounds`, `self.w_i`, `self.w_prior`, `self.w_soc`, `self.w_tb`, `self.w_v`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 6、ループ 1、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `len(self.samples) < 2` を判定し、真なら内部処理を行う。
2. `(z_init, Tb_init)` を返す。
3. 関数 `cost` を定義する。
4. `x0` に `np.array([float(z_init), float(Tb_init)], dtype=float)` の結果を代入する。
5. `bounds` に `[self.soc_bounds, self.tb_bounds]` の結果を代入する。
6. `res` に `minimize(cost, x0, method='L-BFGS-B', bounds=bounds, options=dict(maxiter=80))` の結果を代入する。

7. 条件 `not res.success` を判定し、真なら内部処理を行う。
8. `(z_init, Tb_init)` を返す。
9. `(z0, Tb0)` に `(float(res.x[0]), float(res.x[1]))` の結果を代入する。
10. `(_, zN, TbN)` に `self._simulate(z0, Tb0)` の結果を代入する。
11. `(float(zN), float(TbN))` を返す。

代表コード断片

```
def estimate(self, z_init: float, Tb_init: float) -> Tuple[float, float]:
    if len(self.samples) < 2:
        return z_init, Tb_init

    def cost(x):
        z0, Tb0 = float(x[0]), float(x[1])
        J = self.w_prior * ((z0 - z_init) ** 2 + (Tb0 - Tb_init) ** 2)
        outputs, _, _ = self._simulate(z0, Tb0)
        for (_, meas), (z_pred, Tb_pred, I_pred, V_pred) in zip(self.samples, outputs):
            if meas.soc is not None and math.isfinite(meas.soc):
                J += self.w_soc * (z_pred - meas.soc) ** 2
            if meas.Tb is not None and math.isfinite(meas.Tb):
                J += self.w_tb * (Tb_pred - meas.Tb) ** 2
            if meas.I is not None and math.isfinite(meas.I):
                J += self.w_i * (I_pred - meas.I) ** 2
            if meas.V is not None and math.isfinite(meas.V):
                J += self.w_v * (V_pred - meas.V) ** 2
        return J

    x0 = np.array([float(z_init), float(Tb_init)], dtype=float)
    bounds = [self.soc_bounds, self.tb_bounds]
    res = minimize(cost, x0, method='L-BFGS-B', bounds=bounds, options=dict(maxiter=80))
    if not res.success:
        return z_init, Tb_init
    z0, Tb0 = float(res.x[0]), float(res.x[1])
    _, zN, TbN = self._simulate(z0, Tb0)
    return float(zN), float(TbN)
```

7.8 L89 関数 BatteryMHE.estimate.cost

- 行範囲: L89–L102
- 所属: BatteryMHE.estimate
- 定義: `cost(x)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_simulate`, `float`, `isfinite`, `zip`
- 戻り値の要点: `J`
- 主なローカル代入名: `I_pred`, `J`, `Tb0`, `Tb_pred`, `V_pred`, `_`, `meas`, `outputs`, `z0`, `z_pred`
- 読み取る主なインスタンス属性: `self._simulate`, `self.samples`, `self.w_i`, `self.w_prior`, `self.w_soc`, `self.w_tb`, `self.w_v`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 4、ループ 1、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. (z0, Tb0) に (float(x[0]), float(x[1])) の結果を代入する。
2. J に $\text{self.w_prior} * ((z0 - z_init) ** 2 + (Tb0 - Tb_init) ** 2)$ の結果を代入する。
3. (outputs, _, _) に $\text{self._simulate}(z0, Tb0)$ の結果を代入する。
4. $\text{zip}(\text{self.samples}, \text{outputs})$ を順に走査し、各要素を $((_, \text{meas}), (z_pred, Tb_pred, I_pred, V_pred))$ に入れて処理する。
5. 条件 $\text{meas.soc is not None and math.isfinite}(\text{meas.soc})$ を判定し、真なら内部処理を行う。
6. J を Add で更新する。
7. 条件 $\text{meas.Tb is not None and math.isfinite}(\text{meas.Tb})$ を判定し、真なら内部処理を行う。
8. J を Add で更新する。
9. 条件 $\text{meas.I is not None and math.isfinite}(\text{meas.I})$ を判定し、真なら内部処理を行う。
10. J を Add で更新する。
11. 条件 $\text{meas.V is not None and math.isfinite}(\text{meas.V})$ を判定し、真なら内部処理を行う。
12. J を Add で更新する。
13. J を返す。

代表コード断片

```
def cost(x):
    z0, Tb0 = float(x[0]), float(x[1])
    J = self.w_prior * ((z0 - z_init) ** 2 + (Tb0 - Tb_init) ** 2)
    outputs, _, _ = self._simulate(z0, Tb0)
    for (_, meas), (z_pred, Tb_pred, I_pred, V_pred) in zip(self.samples, outputs):
        if meas.soc is not None and math.isfinite(meas.soc):
            J += self.w_soc * (z_pred - meas.soc) ** 2
        if meas.Tb is not None and math.isfinite(meas.Tb):
            J += self.w_tb * (Tb_pred - meas.Tb) ** 2
        if meas.I is not None and math.isfinite(meas.I):
            J += self.w_i * (I_pred - meas.I) ** 2
        if meas.V is not None and math.isfinite(meas.V):
            J += self.w_v * (V_pred - meas.V) ** 2
    return J
```

8 処理の流れ

1. ソース中の主要関数を通じて処理を進める。

9 このファイルの位置づけ

この資料群では、`mpc_solarcar/estimator.py` を **estimator** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。